第18天 · 看见 MCP 的真实消息往返

观察本地进程如何初始化、列出工具并调用工具。

标准路线：60分钟；忙碌日：20分钟；扩展练习另计。

今天的目标

- 运行真实 stdio 子进程通信。

- 区分初始化请求、通知、工具发现与工具调用。

- 说明教学子集与完整协议实现的区别。

开始之前

- 已经完成第 17 课，或能解释其产物。

- 打开实践包 practice 目录；后文 python 可替换为虚拟环境内解释器的完整路径。

一小时安排

- 15分钟 · 理解概念：阅读本课四节讲解，弄清一个关键机制与适用边界。

- 30分钟 · 亲手操作：先跑基线，再做一次单变量实验，保留前后结果。

- 10分钟 · 检查结果：逐项验收，完成两道自测并阅读解析。

- 5分钟 · 留下记录：写出本课发现、未解决问题和下次第一步。

01 · 理解概念

MCP统一连接方式，不替代工具实现

MCP把工具与数据接口组织成可协商的协议。主机应用、客户端连接器和服务端各有职责；服务端仍需实际执行检索或计算，协议不会自动让工具更正确。

本课启动真实的本地子进程，用标准输入输出传递 JSON-RPC 消息。你会看到 initialize、初始化完成通知、tools/list 与 tools/call。这比只打印一张流程图更接近协议的实际工作方式。

初始化先确认彼此支持什么

客户端不能一上来就假定对端支持所有功能。初始化会交换协议版本和能力，再通知对方初始化完成。本课手写教学子集使用2025-06-18协议版本，只覆盖课程需要的一条路径；可选官方SDK示例另按其支持版本协商，当前验收记录为2025-11-25。

不要把本课脚本称为完整 MCP SDK。协议还涉及错误、取消、传输、授权和不同版本的行为。阅读规范时先把实际出现的字段认出来，其余内容留作扩展清单。

标准输出是协议通道

在 stdio 连接中，stdout 不是普通的调试打印窗口。若服务端把“我启动了”这种日志写进协议通道，客户端可能把它当作 JSON 解析而失败。调试信息应与协议数据分开。

观察 requests、responses 和 stderr。请求ID帮助匹配响应；通知与请求不同，通常不要求对应响应。你应能够解释为什么消息数量不一定一一相等。

官方SDK适合后续扩展

主线使用标准库展示最小通信，便于在普通电脑复现。可选官方Python SDK示例能帮助理解正式客户端和服务端的生命周期。实践包锁定mcp==1.30.0，可运行python optional/mcp_sdk_demo.py；它与手写教学子集是两条独立练习路径。

SDK 的主分支文档可能已经迁移到新主版本。安装前先看 optional 说明，不能把新文档的类名和旧版本示例混用；是否实际运行成功要以自己的日志为准。

02 · 一步一做

命令在解压后的 practice 文件夹执行。Windows可使用 .\.venv\Scripts\python.exe；macOS/Linux可使用 .venv/bin/python 替代 python。

1. 找到本课的输入与配置

为什么做：先知道程序读什么，后面才能解释输出为何改变。

在 VS Code 中打开 experiments.json，定位 day18。用下面的命令确认当前目录正确，并把各参数的作用写在笔记中。

```
python -c "import json; print(json.load(open('experiments.json', encoding='utf-8'))['day18'])"
```

预期结果：终端显示本课配置；键名与文件中的内容一致。

2. 运行本课基线

为什么做：保持默认设置的一次运行，是后续比较的参照。

执行命令，打开新生成的 outputs/day18.json。先检查 day、mode 和 result，不急着修改代码。

```
python run.py day 18
```

预期结果：结果写入 outputs/day18.json；主线 mode 明确为 offline，代表规则教学流程。

3. 沿输出检查机制

为什么做：不能只看程序有没有报错，要检查它实际做了什么。

格式化阅读输出，重点检查requests、responses 与 stderr。把一个输入参数和一个输出字段用箭头对应起来。

```
python -m json.tool outputs/day18.json
```

预期结果：能够定位requests、responses 与 stderr，并用自己的话解释该结果。

4. 改变一个条件并比较

为什么做：主动制造可解释的变化，才能验证自己理解的机制。

完成下面“改一改”实验。对比基线与新结果，记录哪些字段改变、哪些保持稳定；不要同时改模型、材料和参数。

预期结果：留下一组前后结果和一句解释；失败状态也可以是正确的实验产物。

改一改 · 通过工具调用参数控制返回数量

- 先执行 python run.py day 18，将 outputs/day18.json 复制为自己的基线备份。

- 在 VS Code 打开 experiments.json，只修改 day18.top_k 为 1，保持其他键不变并保存。

- 执行 python -m json.tool experiments.json 检查语法，再执行 python run.py day 18。

- 对比两份结果，重点观察requests、responses 与 stderr；解释改变原因后恢复原设置或明确记录保留值。

观察什么：重新运行后，tools/call 的参数和返回检索结果规模应反映 top_k=1；初始化和工具发现仍按原先顺序执行。

机制解释：传输协议与业务参数是不同层面，改变检索规模不需要改变整个连接流程。

常见问题

修改配置后结果没有变化

原因：改了其他天的字段，或编辑后尚未保存。

处理：核对 experiments.json 中的 day18，保存后重新运行；检查实际执行目录。

把受控失败误当成程序坏了

原因：本课实验主动制造了边界条件。

处理：先对照 expected 和状态字段，区分设计内拒绝与未处理异常；保留错误信息再修复。

03 • 验收

观察本地进程如何初始化、列出工具并调用工具。 保留基线、修改结果与自己的解释。

成果文件：outputs/day18.json

☐ 能在输出中识别初始化、工具列表与调用消息。

☐ 解释请求ID、通知和响应数量。

☐ 改变 top_k 后能指出对应的请求与结果变化。

自测与解析

1. stdio服务端应把普通调试日志写到哪里?

- A. 协议stdout中随意插入

- B. stderr等与协议消息分离的通道

- C. 藏进请求ID

- D. 覆盖客户端配置

答案：B。 stdout承载机器可解析消息，混入普通文字会破坏协议读取。

2. 运行本课最小示例成功能证明什么?

- A. 兼容所有MCP功能与版本

- B. 本课覆盖的有限消息流程可以运行

- C. 所有外部工具都安全

- D. 任意网络服务都不需授权

答案：B。 测试结论只能覆盖实际实现和执行的路径。

小词典

- stdio：通过进程标准输入输出交换消息的本地传输方式。

- JSON-RPC：带方法、参数和请求标识的远程过程调用消息格式。

- Capability：连接初始化时声明的功能支持情况。

04 · 我的记录

- 工具没有出现在列表里时，我先检查初始化还是模型提示词?

- 哪些教学实现的缺口在接入外部服务前必须补上?

忙碌日：20分钟

- 用5分钟阅读第一个概念，写出本课要解决的问题。

- 用10分钟运行 python run.py day 18，打开 outputs/day18.json，检查一个关键字段。

- 用5分钟完成第一道自测并留一句解释，完整实验与验收留到下次。

保存一份运行结果和自己的解释；精简练习单独记录，不等于整课验收全部完成。

有余力再做：20分钟选学 · 把本课机制迁移到一个自己的任务

- 从自己的文献整理或数据分析流程中选一个相似环节。

- 写清输入、允许动作、失败出口与验收方法。

- 提出一个会让当前实现失败的反例，并标明它是待实现扩展。

参考资料与阅读范围

MCP：2025-11-25 协议规范：先读 Overview、Base Protocol 与消息生命周期，再查 tools/list、tools/call。
https://modelcontextprotocol.io/specification/2025-11-25

MCP：官方 Python SDK：对照官方SDK的服务器、客户端与版本迁移说明；按实践包锁定版本运行。
https://github.com/modelcontextprotocol/python-sdk

下一步

下一课将相同业务步骤表达为显式的状态图。